

Projet : Une chaîne de vérification pour modèles de procédés

Ingénierie Dirigée par les Modèles

Tristan Gay et Clément Gris - 5 ModIA - Groupe C

2024 – 2025

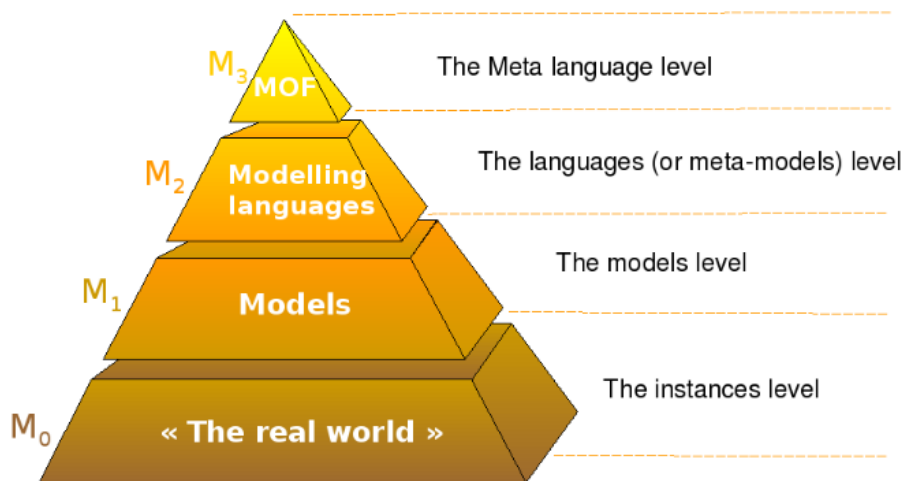


Table des matières

1	Introduction	3
2	Présentation des métamodèles	4
2.1	SimplePDL	4
2.2	PetriNet	5
3	La sémantique statique	7
3.1	La sémantique statique de SimplePDL	7
3.2	La sémantique statique de PetriNet	8
4	Transformation modèle à texte (Acceleo)	10
4.1	SimplePDL vers DOT	10
4.2	PetriNet vers DOT	11
5	Syntaxe concrète graphique (Sirius)	12
6	Transformation modèle à modèle (ATL)	13
6.1	SimplePDL vers TaskMaster	13
6.2	SimplePDL vers PetriNet	13
7	Syntaxe concrète texte (XText)	15
8	Conclusion	18
9	Liste détaillée des fichiers rendus	19
9.1	Eclipse Workspace	19
9.2	Eclipse de déploiement	23

1 Introduction

L'objectif de ce projet est de mettre en place une procédure permettant de vérifier la bonne conception et la terminaison des modèles de procédés. Ce projet repose sur l'utilisation de deux métamodèles principaux, SimplePDL et PetriNet. Les outils mis en place utilisent Aceleo pour la transformation de modèles en texte, ATL pour la transformation de modèle à modèle, et XText pour la génération de modèles sous forme textuelle. L'ensemble du développement a été réalisé sur Eclipse.

En intégrant ces outils, nous nous assurons que les utilisateurs peuvent manipuler les modèles de manière intuitive, même sans connaissances techniques approfondies. Le projet vise ainsi à rendre accessibles des processus complexes tout en offrant des mécanismes de vérification qui permettent d'identifier rapidement toute erreur de conception ou problème de terminaison, et ce, sans nécessiter une expertise en informatique.

2 Présentation des métamodèles

2.1 SimplePDL

Nous nous intéressons à des modèles de processus composés d'activités et de dépendances entre ces activités. L'objectif est de pouvoir vérifier la cohérence et la terminaison d'un modèle de processus.

Le modèle de procédé utilisé provient du TD réalisé en classe. La figure 1 donne un exemple de processus qui comprend quatre activités : Conception, RédactionDoc, Programmation et RédactionTests. Les activités sont représentées par des ellipses. Les flèches entre activités représentent les dépendances. Une étiquette sur les arcs (ou flèches) nous permet de voir les actions à réaliser afin de pouvoir réaliser l'activité liée. On ne peut donc commencer la RédactionDoc que si la Conception est commencée et la terminer que si la Conception est terminée, à titre d'exemple.

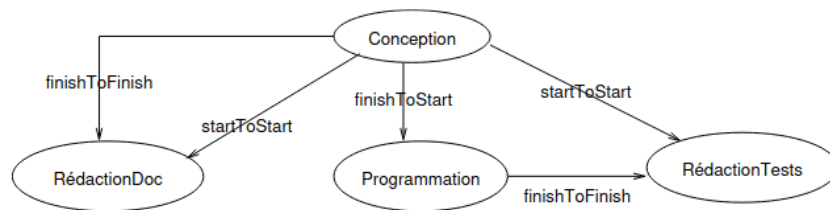


FIG. 1 : Exemple de modèle de procédé issu du TD

Dans un premier temps, nous partons du métamodèle suivant :

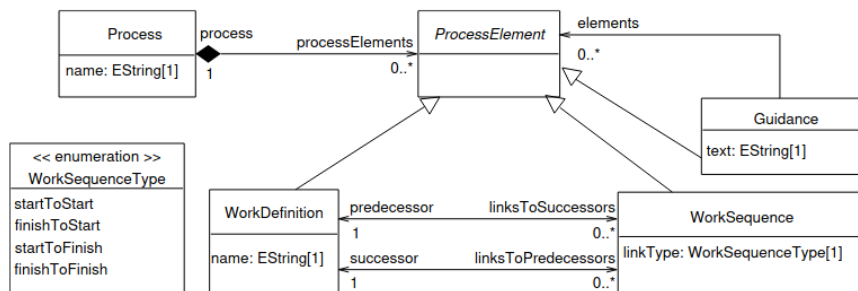


FIG. 2 : Métamodèle SimplePDL fournit dans le cours

Nous complétons ce métamodèle afin qu'il prenne en compte les Ressources :

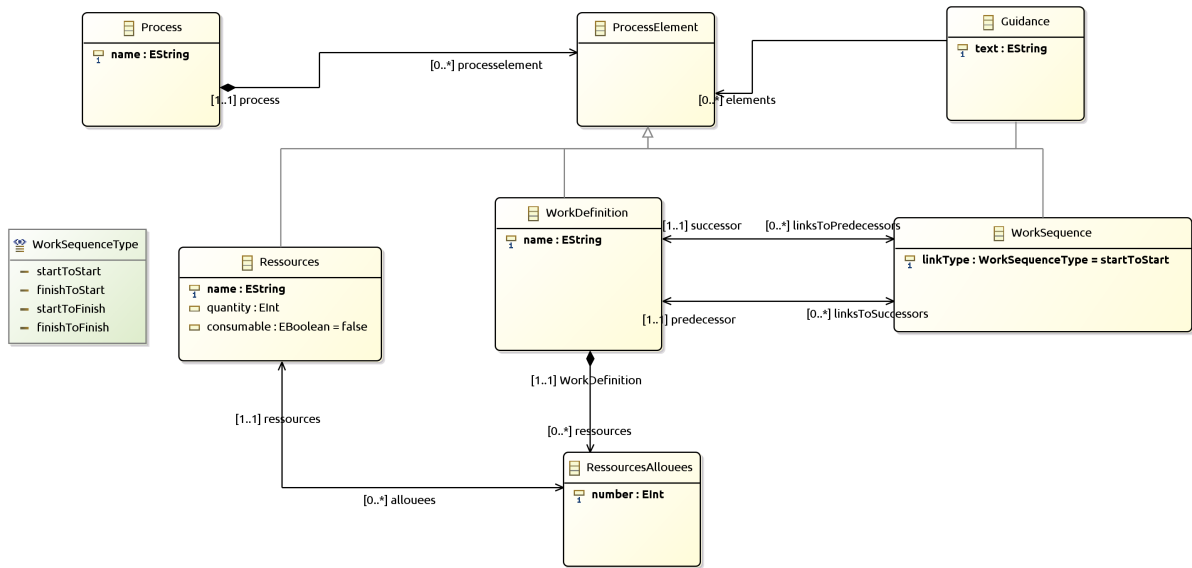


FIG. 3 : SimplePDL

Nous avons ajouté à SimplePDL deux classes supplémentaires : Ressource et RessourceAllouée. Ressource définit les différentes ressources disponibles pour les processus. Elle hérite de ProcessElement, car c'est un élément du processus. Elle possède comme attributs un nom, une quantité, et une indication précisant si la ressource est consommable ou non.

La classe RessourceAllouée hérite de Ressource. Elle possède un attribut indiquant le nombre de ressources allouées. Une Ressource peut allouer de 1 à n ressources, mais une RessourceAllouée ne provient que d'une seule Ressource.

On peut également observer qu'une WorkDefinition peut posséder ou non des RessourcesAllouées.

2.2 PetriNet

La classe PetriNet permet de modéliser un réseau de Petri. Un réseau de Petri est un modèle mathématique utilisé pour représenter et analyser les comportements dans les systèmes informatiques ou industriels.

Nous avons construit notre métamodèle comme suit :

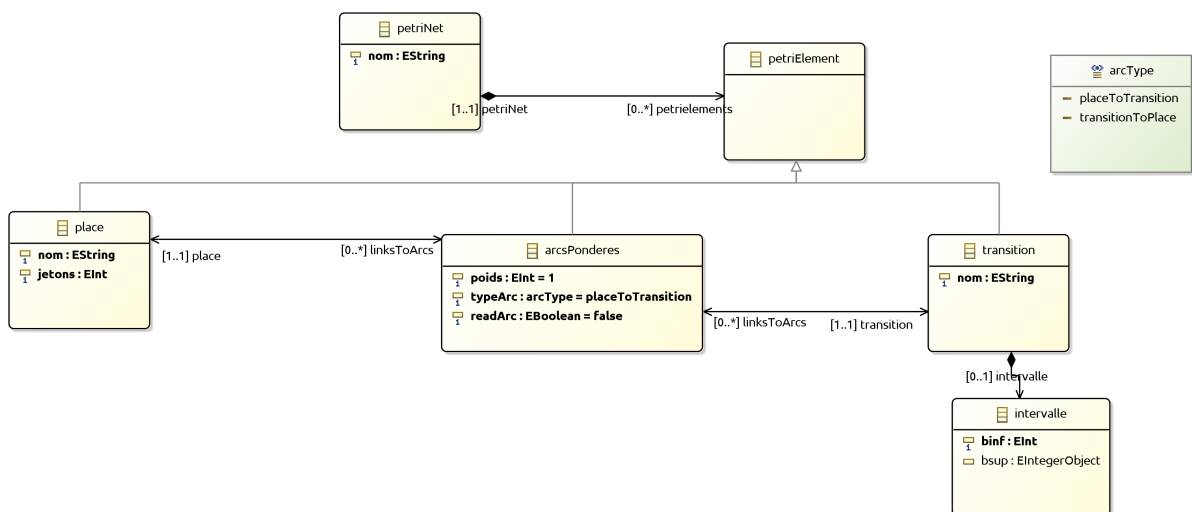


FIG. 4 : Diagramme de classes du modèle PetriNet

Un `PetriNet` se compose de 0 à n `PetriElement`.

Une `Place` est définie par un nom et un nombre de jetons.

Une `Transition` est définie par un nom. Elle peut éventuellement posséder un intervalle, défini par une borne inférieure et une borne supérieure (facultative).

Les `Place` et les `Transition` sont reliées par des `ArcPondéré`, qui possèdent les attributs suivants :

- un `poids`;
- un `typeArc` (`placeToTransition` ou `transitionToPlace`);
- un `readArc` (`true` ou `false`);

Ainsi, avec ce métamodèle, nous respectons les contraintes des réseaux de Petri, telles que l'interdiction de relier directement deux `Place` ou deux `Transition`, par exemple.

3 La sémantique statique

Lorsque nous définissons un métamodèle avec un fichier *ecore*, nous voulons que les modèles créés ensuite soient conformes à ce métamodèle. Pour cela, dans le fichier *.xmi* du modèle, nous pouvons faire un clic droit, puis *validate* pour vérifier le modèle. Le problème, c'est que dans certains cas, d'autres règles doivent également être vérifiées pour être conforme au métamodèle. Il faut donc définir la sémantique statique du métamodèle. Au travers de plusieurs exemples de modèles, nous allons voir l'utilité de cette sémantique pour les métamodèles *SimplePDL* et *PetriNet*.

3.1 La sémantique statique de SimplePDL

Le premier cas que nous allons voir, c'est celui d'un réseau ne vérifiant pas les règles du fichier *.ecore*. Considérons le modèle de la figure 5. Sa *WorkSequence* ne possède pas de Successeur, ce qui soulève une erreur, comme l'indique le logo rouge sur la figure.

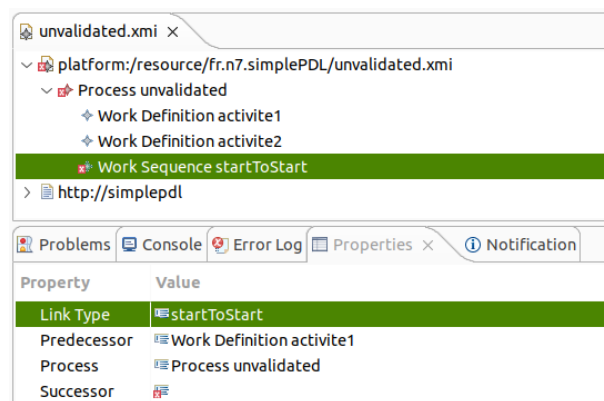


FIG. 5 : Modèle non valide

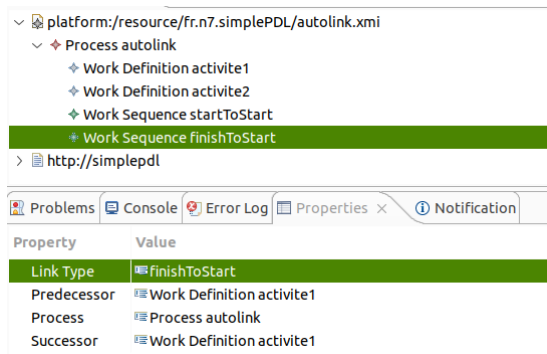
Nous allons maintenant voir plusieurs exemples qui sont conformes au modèle *ecore*, mais qui posent problème, montrant l'intérêt de la sémantique statique.

Le premier cas, c'est celui où plusieurs *Work Definitions* ont le même nom, comme pour le modèle présenté dans la figure 6a. Une bonne sémantique statique permet de relever ce problème lors de la validation, comme le montre la figure 6b.



FIG. 6 : Validation d'un modèle SimplePDL avec noms non uniques

Le deuxième cas, présenté dans la figure 7, est un process avec une *Work Sequence* qui a un prédécesseur et un successeur identiques, ce qui n'est pas autorisé. Ainsi, il est important que notre sémantique nous permette de soulever cette erreur lors de la validation, comme dans la figure 7b.



(a) Modèle SimplePDL avec un lien vers lui même

```
Résultat de validation pour autolink.xml:
- Process: OK
- WorkDefinition: OK
- WorkSequence: 1 erreurs trouvées
=> Erreur dans simplePDLImpl.WorkSequenceImpl@5af3af09 (linkType: finishToStart): La dépendance relie l'activité activite1 à elle-même
- Guidance: OK
- Ressources: OK
Fin.
```

(b) Validation du modèle avec un lien vers lui même

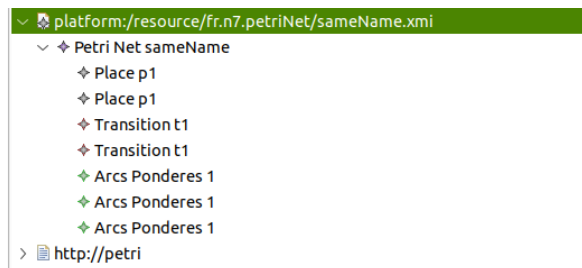
FIG. 7 : Validation d'un modèle SimplePDL avec une boucle

3.2 La sémantique statique de PetriNet

Dans cette section, nous effectuons la même chose que précédemment pour SimplePDL. Si nous nous plaçons dans un cas où un arc n'est pas relié à une Place, le modèle ne sera pas Validé et lèvera donc une erreur.

Nous avons envisagé tous les autres cas possibles qui pourraient poser problème pour notre modèle, mais qui seraient néanmoins considérés comme Validé dans Eclipse :

- Si deux Place ou deux Transition portent le même nom, nous avons fixé une contrainte afin qu'une erreur soit levée.

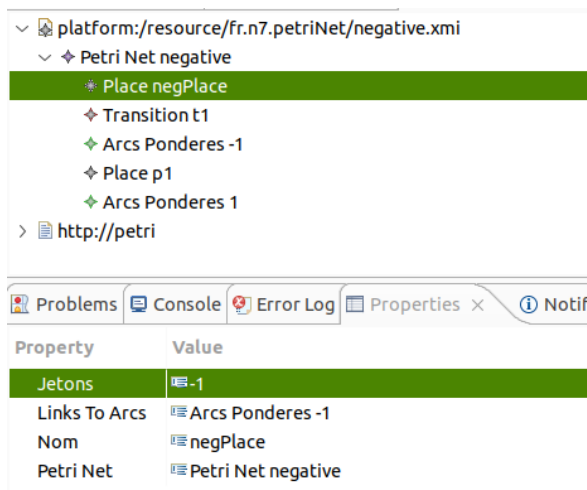


```
Résultat de validation pour sameName.xml:
- Petri Net: OK
- Place: 2 erreurs trouvées
=> Erreur dans pl [petri.impl.placeImpl@d9b7cce (nom: p1, jetons: 1)]: Le nom de la place (p1) n'est pas unique
=> Erreur dans pl [petri.impl.placeImpl@8dbdacl (nom: p1, jetons: 0)]: Le nom de la place (p1) n'est pas unique
- Transition: 2 erreurs trouvées
=> Erreur dans t1 [petri.impl.transitionImpl@66e28b53a (nom: t1)]: Le nom de la transition (t1) n'est pas unique
=> Erreur dans t1 [petri.impl.transitionImpl@71889907 (nom: t1)]: Le nom de la transition (t1) n'est pas unique
- Arc Pondéré: OK
- Intervalle: OK
```

FIG. 9 : Contrainte pour le conflit de nom

FIG. 8 : Pas d'erreur levée par Eclipse pour le nom

- Si une Place possède un nombre de jetons négatif ou si un Arc Pondéré a un poids négatif.



```

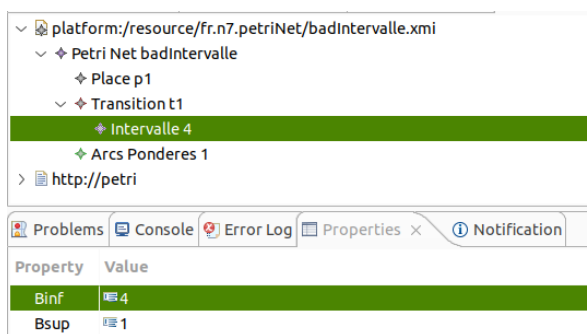
Résultat de validation pour negative.xml:
- Petri Net: OK
- Place: 1 erreurs trouvées
=> Erreur dans negPlace [petriImpl.placeImpl@3c3e389 (nom: negPlace, jetons: -1)]: La place (negPlace) a un nombre de jetons négatif.
- Transition: OK
- Arc Ponderé: 1 erreurs trouvées
=> Erreur dans petriImpl.arcsPonderesImpl@6aba2b86 (poids: -1, typeArc: placeToTransition, readArc: false): un arc a un poids négatif.
- Intervalle: OK

```

FIG. 11 : Contrainte sur le poids négatif sur un arc

FIG. 10 : Pas d'erreur levée par Eclipse pour le poids négatif

- Si un intervalle est incorrect (par exemple si borneSup < borneInf).



```

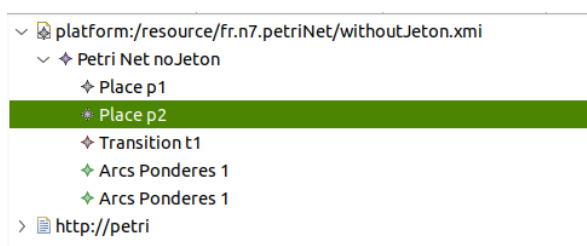
-----
Résultat de validation pour badIntervalle.xml:
- Petri Net: OK
- Place: OK
- Transition: OK
- Arc Ponderé: OK
- Intervalle: 1 erreurs trouvées
=> Erreur dans petriImpl.intervalleImpl@1580a8e (binf: 4, bsup: 1): La transition a un intervalle avec une borne inférieure supérieure à la borne supérieure

```

FIG. 13 : Contrainte sur l'intervalle

FIG. 12 : Pas d'erreur levée par Eclipse sur l'intervalle

- Si le réseau de Petri ne contient aucun jeton.



```

-----
Résultat de validation pour withoutJeton.xml:
- Petri Net: OK
- Place: 2 erreurs trouvées
=> Erreur dans p1 [petriImpl.placeImpl@74e32983 (nom: p1, jetons: 0)]: Le réseau de Petri ne contient aucun jeton : au moins une place doit avoir un jeton.
=> Erreur dans p2 [petriImpl.placeImpl@74e32983 (nom: p2, jetons: 0)]: Le réseau de Petri ne contient aucun jeton : au moins une place doit avoir un jeton.
- Transition: OK
- Arc Ponderé: OK
- Intervalle: OK

```

FIG. 15 : Contrainte pour un manque de jeton

FIG. 14 : Pas d'erreur levée par Eclipse pour un réseau sans jeton

4 Transformation modèle à texte (Acceleo)

Acceleo permet d'effectuer une transformation d'un modèle vers un texte. Il nous permet de générer des fichiers HTML ou encore des fichiers DOT que nous manipulerons par la suite. Une fois au format DOT, il nous est possible de convertir le fichier en PDF afin de le visualiser. Ce sont les fichiers MTL exécutés avec Acceleo qui permettent la transformation de modèle à texte.

4.1 SimplePDL vers DOT

Le fichier toHTML.mlt permet de générer un fichier HTML à partir d'un modèle de processus, en utilisant le nom du processus comme titre et nom de fichier. Il affiche la liste des activités (WorkDefinitions) du processus, en indiquant leurs dépendances avec d'autres activités via des phrases comme « A requires B to be started/finished ».

Les requêtes définissent comment extraire les WorkDefinitions du processus et comment traduire les types de liens (WorkSequenceType) en états lisibles ("started" ou "finished").

Process "model"

Work definitions

- Programmation requires Conception to be finished.
- RedactionTests requires Conception to be started, Programmation to be finished.
- RedactionDoc requires Conception to be started, Conception to be finished.
- Conception

FIG. 16 : SimplePDL en HTML

Puis, nous générons un fichier .dot qui représente un processus sous forme de graphe orienté pour être visualisé avec Graphviz. Chaque activité (WorkDefinition) est un noeud, et les dépendances entre elles (WorkSequence) sont représentées par des flèches étiquetées selon le type de lien (par exemple s2s, f2s).

Les requêtes extraient les activités du processus et traduisent les types de liens en étiquettes lisibles dans le format .dot.

```
4 nodes | 5 edges
1= digraph model {
2   Conception -> Programmation [arrowhead=vee label=f2s]
3   Conception -> RedactionTests [arrowhead=vee label=s2s]
4   Programmation -> RedactionTests [arrowhead=vee label=f2f]
5   Conception -> RedactionDoc [arrowhead=vee label=s2s]
6   Conception -> RedactionDoc [arrowhead=vee label=f2f]
7 }
8
```

FIG. 17 : SimplePDL to DOT

Enfin, avec la commande `dot fichier.dot -Tpdf -o fichier.pdf`, nous convertissons notre fichier dot au format pdf.

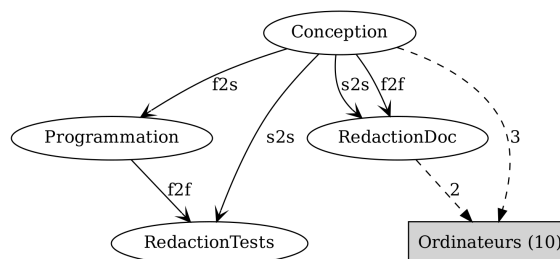


FIG. 18 : SimplePDL en PDF

4.2 PetriNet vers DOT

De la même manière que pour SimplePDL, nous avons généré un fichier .dot représentant un réseau de Petri sous forme de graphe orienté. Il transforme chaque place en cercle (affichant son nom et le nombre de jetons) et chaque transition en boîte grise (avec un intervalle si défini).

On ajoute les arcs entre places et transitions en tenant compte du type (normal ou "read arc") et du poids, avec des styles et étiquettes différentes.

```

10 nodes | 14 edges
1 digraph ifip {
2   rankdir=LR;
3
4   // Places
5   p1 [label="p1\n(1)", shape=circle];
6   p2 [label="p2\n(2)", shape=circle];
7   p3 [label="p3\n(0)", shape=circle];
8   p4 [label="p4\n(0)", shape=circle];
9   p5 [label="p5\n(0)", shape=circle];
10
11  // Transitions
12  t1 [
13    label="t1\n[4,9]",
14    shape=box, style=filled, fillcolor=gray
15  ];
16  t2 [
17    label="t2",
18    shape=box, style=filled, fillcolor=gray
19  ];
20  t3 [
21    label="t3\n[1,w]",
22    shape=box, style=filled, fillcolor=gray
23  ];
24  t4 [
25    label="t4",
26    shape=box, style=filled, fillcolor=gray
27  ];
28  t5 [
29    label="t5",
30    shape=box, style=filled, fillcolor=gray
31  ];
32
33  // Arcs
34
35  t3 -> p2;
36  p5 -> t3;
37  p5 -> t4 [label="1", style=dashed];
38  p3 -> t4;
39  t4 -> p3;
40  p3 -> t5;
41  t5 -> p1;
42  n1 -> t1;

```

FIG. 19 : PetriNet to DOT

Et comme avant, on peut le convertir en PDF.

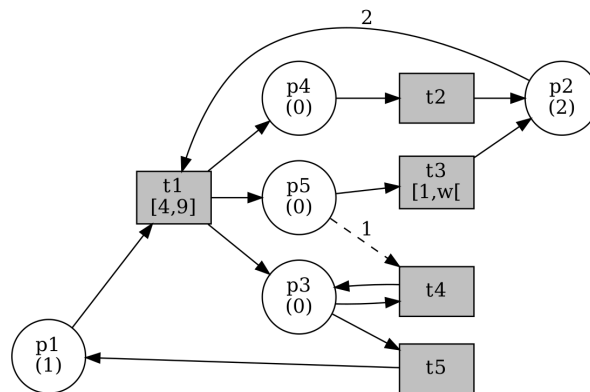


FIG. 20 : PetriNet en PDF

5 Syntaxe concrète graphique (Sirius)

Sirius est un outil intégré à l'écosystème Eclipse Modeling, qui permet de créer des éditeurs graphiques pour des langages de modélisation. Il repose sur les technologies EMF (Eclipse Modeling Framework) et GMF (Graphical Modeling Framework). Grâce à Sirius, il est possible de définir une syntaxe concrète graphique pour un métamodèle décrit en Ecore, et de générer automatiquement un éditeur graphique intégré à Eclipse. Cet éditeur permet de visualiser et de manipuler les modèles de manière plus intuitive via des formes (nœuds, arcs, annotations...) et une palette d'outils de création.

Nous avons travaillé sur un fichier **.odesign**, permettant de définir les représentations graphiques de chaque élément, et également la palette d'outils. Cette palette d'outils permet de créer directement des éléments depuis le graphique.

Le résultat final est un éditeur graphique complet pour le langage SimplePDL, permettant de créer et de visualiser graphiquement des processus composés d'activités (*WorkDefinition*), de dépendances (*WorkSequence*) et de commentaires (*Guidance*). Chaque activité est représentée sous forme de losange gris, les dépendances par des flèches dont le style reflète le type de lien, et les commentaires sous forme de notes jaunes reliées aux éléments décrits. Un exemple de cette représentation pour un modèle nommé *developpement* est présenté dans la figure 21. On y voit bien les différents styles et représentations en fonction des cas.

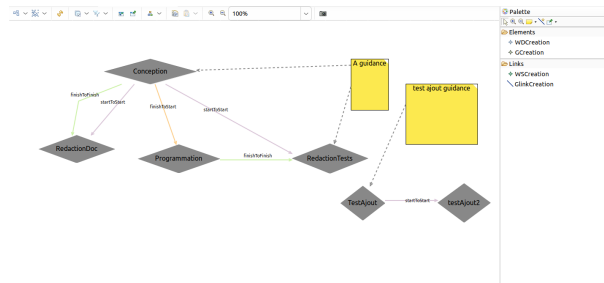


FIG. 21 : Représentation Sirius pour le modèle *developpement*

L'utilisateur peut créer de nouveaux éléments directement via la palette d'outils, en cliquant dans le diagramme. L'éditeur reste synchronisé avec le modèle ecore et permet d'enrichir celui-ci depuis le graphique. Cette palette apparaît à droite de la figure 21.

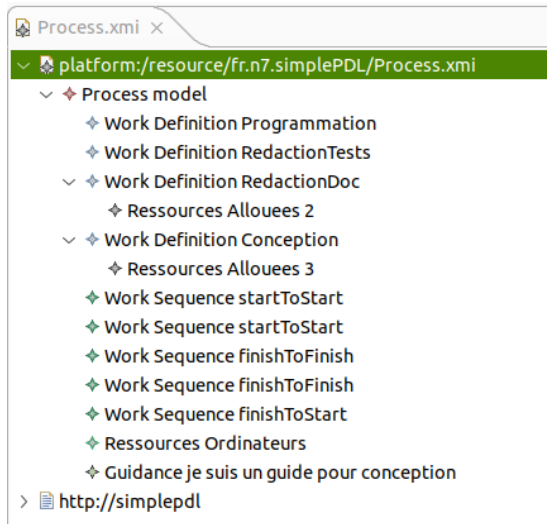
6 Transformation modèle à modèle (ATL)

ATL est un outil de transformation de modèles en modèles conçu. Il permet de définir des transformations à l'aide de règles déclaratives : au lieu de décrire chaque étape en détail, on indique simplement comment un type d'objet doit être transformé. ATL s'occupe ensuite automatiquement de la gestion des modèles, des ressources et de l'exécution des règles dans le bon ordre et au bon moment.

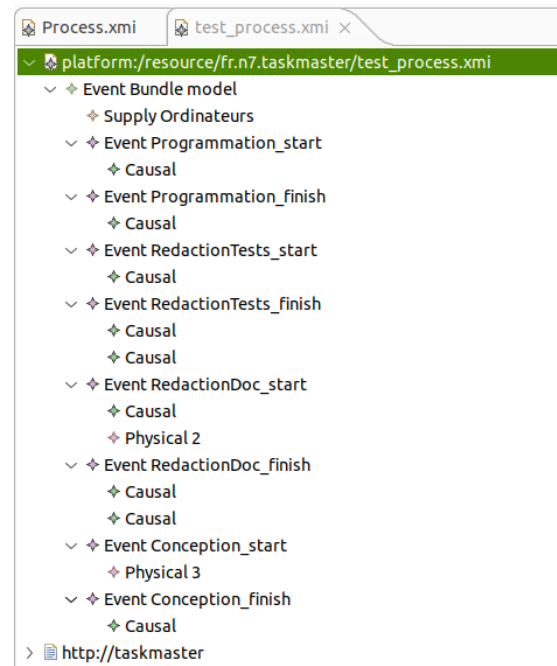
6.1 SimplePDL vers TaskMaster

La transformation ATL de SimplePDL vers TaskMaster permet de convertir deux modèles SimplePDL en un modèle TaskMaster, en produisant une sortie conforme au métamodèle TM.

- Chaque élément **Process** du modèle source est transformé en un **EventBundle** dans le modèle cible, regroupant les événements associés à ce processus.
- Chaque **WorkDefinition** est convertie en deux événements distincts (un pour le début et un pour la fin), reliés entre eux par une relation de causalité indiquant que l'événement de fin dépend de celui de début.
- Les **WorkSequence** sont traduites en contraintes de causalité entre événements, en fonction de leur type de lien (**start-to-start**, **finish-to-start**, etc.).
- Les ressources (**Ressources**) et les allocations de ressources (**RessourcesAllouees**) sont respectivement converties en objets **Supply** et **Physical**, représentant la disponibilité et l'utilisation effective des ressources au sein du processus.



(a) SimplePDL.xmi



(b) TaskMaster.xmi

FIG. 22 : Transformation de SimplePDL vers TaskMaster

6.2 SimplePDL vers PetriNet

La seconde transformation ATL, `simplepdltopetri.atl`, permet de convertir un modèle SimplePDL en un modèle de réseau de Petri, en générant un élément `petriNet` pour chaque `Process`.

- Chaque `WorkDefinition` est traduite en trois places (`ready`, `started`, `finished`), deux transitions (`start`, `finish`) et quatre arcs connectant ces éléments entre eux.
- La place `ready` est initialisée avec un jeton, indiquant que l'activité correspondante peut débuter, tandis que les places `started` et `finished` sont vides au départ.
- Chaque `WorkSequence` est transformée en un arc de type `readArc`, exprimant une contrainte de précedence entre deux activités, en fonction du type de lien (`start-to-start`, `finish-to-finish`, etc.).
- Des helpers et des règles spécifiques assurent la correspondance automatique entre les éléments du modèle `SimplePDL` et ceux du réseau de Petri, en prenant en compte le contexte de chaque élément manipulé.

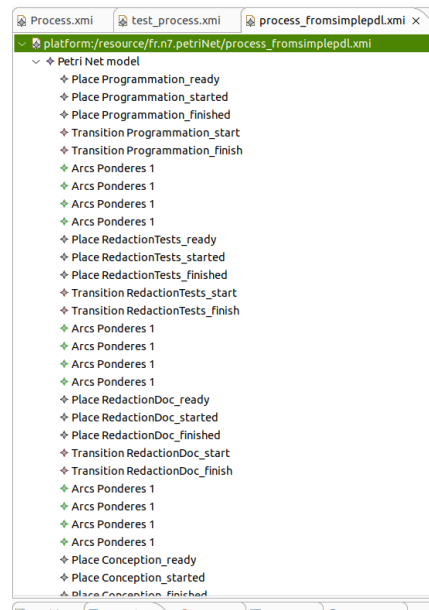


FIG. 23 : PetriFromSimplePDL.xmi

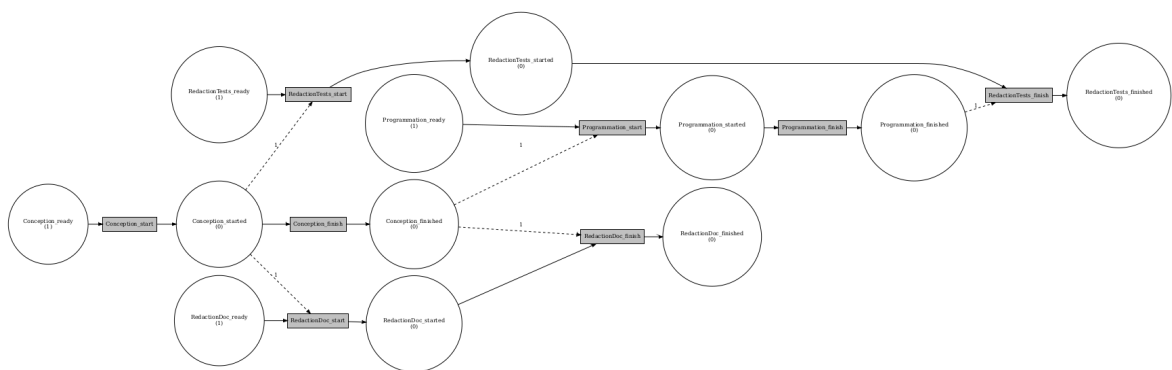


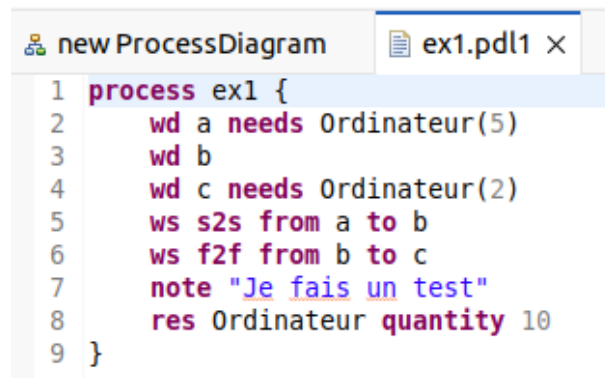
FIG. 24 : PetriFromSimplePDL en PDF

7 Syntaxe concrète texte (XText)

Xtext est un framework de développement open source proposé par Eclipse, destiné à la définition de langages spécifiques (DSL) textuels. Il permet de spécifier à la fois la syntaxe concrète (sous forme de grammaire) et la structure abstraite (métamodèle EMF) du langage. Une fois la grammaire définie, Xtext génère automatiquement un environnement de développement complet, comprenant un éditeur avec coloration syntaxique, complétion, validation, navigation, etc.

Xtext s'intègre parfaitement avec EMF (Eclipse Modeling Framework), permettant de générer des modèles conformes à un métamodèle et manipulables via les outils classiques de la plateforme Eclipse.

Dans le cadre du projet, nous avons défini un langage nommé PDL1 à l'aide de Xtext. Ce langage permet de modéliser des processus SimplePDL. Nous avons créé un projet Xtext (`fr.n7.pdl1`), pour avoir une syntaxe textuelle personnalisée, comme présenté dans la figure 25. C'est dans le fichier `PDL1.xtext`, que sont définies les règles de production pour les différents éléments syntaxiques du modèle.



```
1 process ex1 {
2     wd a needs Ordinateur(5)
3     wd b
4     wd c needs Ordinateur(2)
5     ws s2s from a to b
6     ws f2f from b to c
7     note "Je fais un test"
8     res Ordinateur quantity 10
9 }
```

FIG. 25 : Représentation textuelle pdl1

Une fois la grammaire définie, l'exécution du fichier `GeneratePDL1.mwe2` permet de générer l'infrastructure associée. Un nouvel éditeur textuel est alors utilisable dans Eclipse. Il propose :

- La coloration syntaxique,
- La complétion automatique,
- L'outline structuré du modèle,
- La détection des erreurs syntaxiques.

Les fichiers `.pdl1` peuvent ensuite être ouverts soit avec l'éditeur dédié, soit avec un éditeur EMF pour voir le modèle avec son arborescence, comme dans la figure 26.

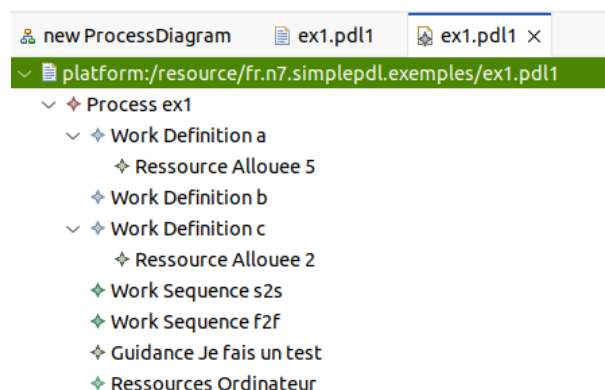


FIG. 26 : Visualisation de ex1.pdl1 avec EMF

Un métamodèle correspondant est automatiquement engendré dans le dossier `model/generated`. Il s'agit d'un fichier PDL1. ecore qui respecte la structure définie dans la grammaire. Ce métamodèle permet d'utiliser les modèles textuels dans les autres outils de la chaîne EMF. Nous pouvons visualiser ce métamodèle dans la figure 27. Nous pouvons voir qu'il est très similaire à celui de SimplePDL (figure 3), mais il y a quelques petites différences.

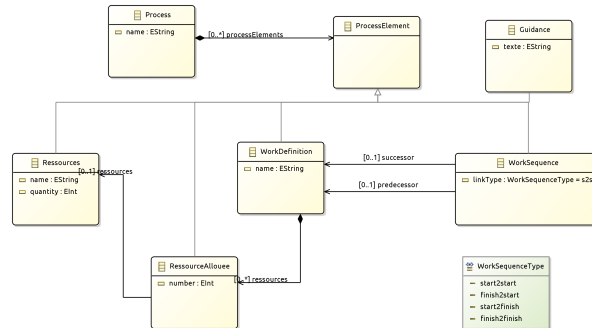


FIG. 27 : Métamodèle PDL1

Pour le projet, nous avons aussi créé une deuxième syntaxe (PDL2) et une troisième (PDL3) avec des grammaires différentes. Ces deux syntaxes ne prennent pas en compte les ressources. Dans la figure 28 (resp 29), nous pouvons voir un exemple de représentation textuelle d'un modèle et le métamodèle de PDL2 (resp PDL3).

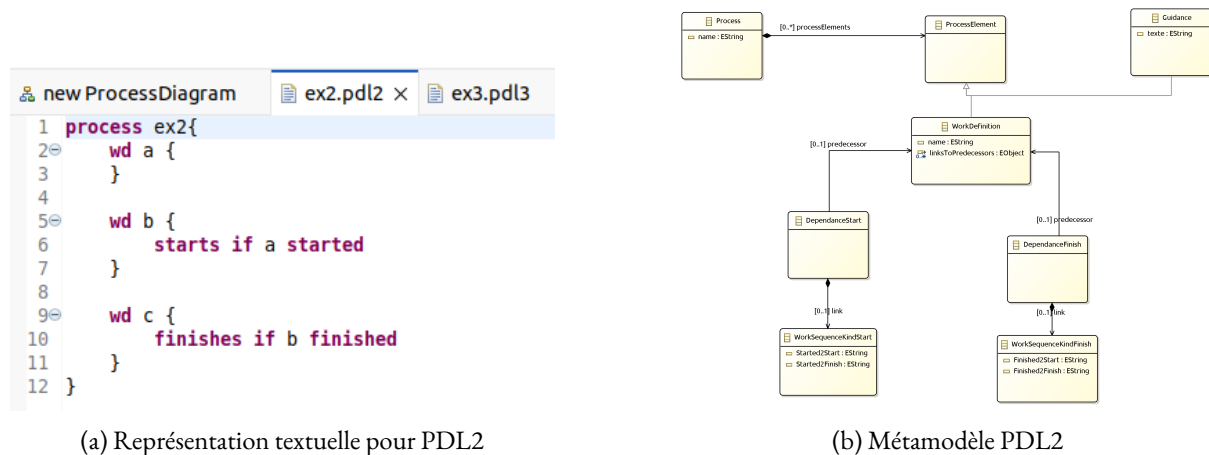


FIG. 28 : Langage PDL2 : syntaxe textuelle et métamodèle

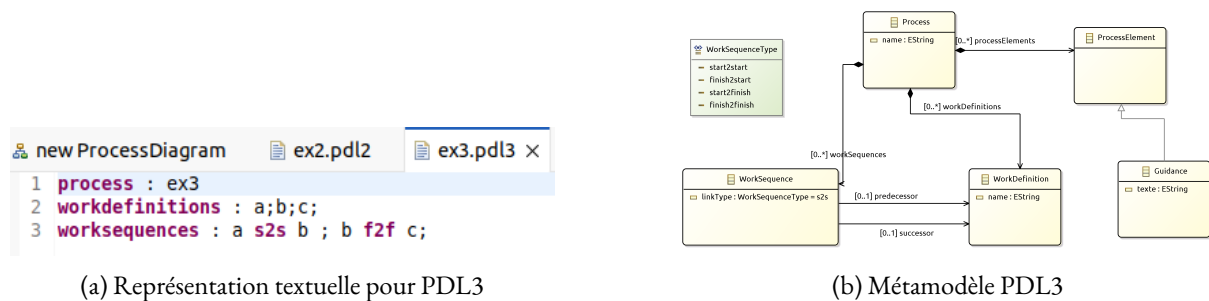


FIG. 29 : Langage PDL3 : syntaxe textuelle et métamodèle

Nous pouvons voir sur ces figures que les métamodèles construits à partir du même modèle sont différents entre eux et différents de PDL1. En plus, ces deux métamodèles sont plus éloignés de SimplePDL.

Les fichiers produits avec l'éditeur généré par Xtext ne sont pas directement compatibles avec le métamodèle SimplePDL. Cela est vrai non seulement pour des langages assez différents comme PDL2, mais aussi pour des langages plus proches comme PDL1. Pour pouvoir exploiter les outils conçus autour de SimplePDL, il est donc nécessaire de transformer ces modèles. Cette transformation peut être réalisée à l'aide d'une transformation model-to-model, par exemple en utilisant le langage ATL. Nous effectuons cette transformation pour le langage PDL1. Nous testons cette transformation sur le modèle de la figure 25, et nous obtenons le modèle de la figure 30.

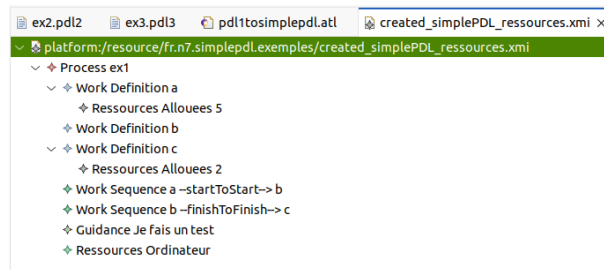


FIG. 30 : Modèle SimplePDL issu d'un modèle PDL1

8 Conclusion

En conclusion, ce projet nous a permis de développer une chaîne de vérification complète pour les modèles de procédés en utilisant les métamodèles SimplePDL et PetriNet. Grâce à des outils comme Acceleo, ATL, Sirius et XText, nous avons réussi à transformer et visualiser les modèles de manière efficace.

L'intégration de ces outils a rendu accessible la manipulation de processus complexes, même pour des utilisateurs sans connaissances techniques approfondies. Les mécanismes de vérification que nous avons mis en place permettent d'identifier rapidement les erreurs de conception ou les problèmes de terminaison, assurant ainsi la robustesse et la fiabilité des modèles.

La définition de la sémantique statique pour les métamodèles a été cruciale pour garantir la conformité des modèles créés, en détectant des erreurs qui ne seraient pas identifiées par les outils standards. Les transformations modèle à texte et modèle à modèle se sont avérées très utiles pour générer des représentations variées et exploitables des modèles, facilitant ainsi leur analyse et leur compréhension.

Enfin, l'éditeur graphique développé avec Sirius offre une interface pour la création et la manipulation des modèles, ce qui améliore considérablement l'expérience utilisateur.

9 Liste détaillée des fichiers rendus

Au vu du grand nombre de fichiers rendus, tous ne sont pas cités ici. Nous mettons tous les projets, et parlons uniquement des fichiers sur lesquels nous avons directement travaillé.

9.1 Eclipse Workspace

- `fr.n7.eclipse.plugin.exemple`
- `fr.n7.eclipse.plugin.exemple.edit`
- `fr.n7.eclipse.plugin.exemple.editor`
- `fr.n7.pdl1`
 - `src/fr.n7/GeneratePDL1.mwe2`
Script MWE2 pour générer l'infrastructure EMF/Xtext.
 - `src/fr.n7/PDL1.xtext`
Définition de la grammaire textuelle du langage.
 - `model/generated/PDL1.aird`
Fichier de représentation graphique.
 - `model/generated/PDL1.ecore`
Métamodèle Ecore du langage.
- `fr.n7.pdl1.ide`
- `fr.n7.pdl1.tests`
- `fr.n7.pdl1.ui`
- `fr.n7.pdl1.ui.tests`
- `fr.n7.pdl2`
 - `src/fr.n7/GeneratePDL2.mwe2`
Script MWE2 pour générer l'infrastructure EMF/Xtext.
 - `src/fr.n7/PDL2.xtext`
Définition de la grammaire textuelle du langage.
 - `model/generated/PDL2.aird`
Fichier de représentation graphique.
 - `model/generated/PDL2.ecore`
Métamodèle Ecore du langage.
- `fr.n7.pdl2.ide`
- `fr.n7.pdl2.tests`
- `fr.n7.pdl2.ui`
- `fr.n7.pdl2.ui.tests`
- `fr.n7.pdl3`
 - `src/fr.n7/GeneratePDL3.mwe2`
Script MWE2 pour générer l'infrastructure EMF/Xtext.

- **src/fr.n7/PDL3.xtext**
Définition de la grammaire textuelle du langage.
- **model/generated/PDL3.aird**
Fichier de représentation graphique.
- **model/generated/PDL3.ecore**
Métamodèle Ecore du langage.
- **fr.n7.pdl3.ide**
- **fr.n7.pdl3.tests**
- **fr.n7.pdl3.ui**
- **fr.n7.pdl3.ui.tests**
- **fr.n7.petriNet**
 - **src/petri.validation/petriValidator.java**
Classe principale pour valider les modèles de réseau de Petri.
 - **src/petri.validation/Utils.java**
Fonctions utilitaires utilisées pendant la validation des modèles.
 - **src/petri.validation/ValidatePetri.java**
Point d'entrée pour exécuter la validation sur un fichier PetriNet.
 - **src/petri.validation/ValidationResult.java**
Structure représentant les résultats de la validation d'un modèle.
 - **badIntervalle.xmi**
Exemple de modèle contenant un intervalle invalide pour test.
 - **ex_dot_acceleo.xmi**
Modèle utilisé comme entrée pour la génération d'un fichier .dot.
 - **negative.xmi**
Modèle avec un marquage initial négatif pour tester les contraintes.
 - **PetriNet.aird**
Fichier contenant les représentations graphiques du PetriNet.
 - **PetriNet.ecore**
Définition du métamodèle EMF du réseau de Petri.
 - **PetriNet.genmodel**
Modèle de génération de code EMF basé sur PetriNet.ecore.
 - **process_fromsimplepdl.xmi**
Modèle PetriNet généré automatiquement depuis un processus SimplePDL.
 - **saisons.xmi**
Exemple de modèle illustrant un processus cyclique.
 - **sameName.xmi**
Exemple de modèle contenant des éléments avec des noms identiques.
 - **unvalidated.xmi**
Modèle volontairement incorrect, servant à tester la validation.
 - **withoutJeton.xmi**
Modèle de test où toutes les places sont vides au départ.
- **fr.n7.petriNet.edit**

- `fr.n7.petriNet.editor`
- `fr.n7.petrinet.exemples`
 - **My.petri**
Fichier décrivant un réseau de Petri, utilisé pour modéliser des processus concurrents.
- `fr.n7.petrinet.todot`
 - **src/fr.n7.petrinet.todot.main/ToDot.java**
Classe Java utilisée pour lancer la transformation d'un modèle Petri en .dot.
 - **src/fr.n7.petrinet.todot.main/ToDot.mtl**
Fichier Aceleo définissant la génération du code DOT à partir d'un modèle PetriNet.
 - **ifip.dot**
Fichier DOT généré à partir d'un modèle, représentant un graphe.
 - **ifip.pdf**
Visualisation du fichier ifip.dot exportée en PDF.
 - **model.dot**
Fichier DOT généré automatiquement depuis un modèle PetriNet.
 - **model.pdf**
Version PDF du graphe correspondant à model.dot.
 - **saisons.dot**
Fichier DOT représentant un modèle de type cyclique (saisons).
 - **saisons.pdf**
Version visuelle du modèle saisons.dot exportée en PDF.
- `fr.n7.simplePDL`
 - **src/simplepdl.manip/SimplePDLCreator.java**
Classe Java permettant de créer un modèle SimplePDL de manière programmatique.
 - **src/simplepdl.manip/SimplePDLManipulator.java**
Classe Java pour modifier dynamiquement un modèle SimplePDL existant.
 - **src/simplepdl.validation/SimplePDLValidator.java**
Classe principale chargée de valider les contraintes sur un modèle SimplePDL.
 - **src/simplepdl.validation/Utils.java**
Fonctions utilitaires partagées par les classes de validation SimplePDL.
 - **src/simplepdl.validation/ValidateSimplepdl.java**
Classe exécutant la validation sur un modèle SimplePDL depuis le main.
 - **src/simplepdl.validation/ValidationResult.java**
Structure de données représentant le résultat de la validation.
 - **models/SimplePDLCreator_Created_Process.xmi**
Modèle SimplePDL généré automatiquement par SimplePDLCreator.
 - **autolink.xmi**
Exemple de modèle erroné avec un lien pointant sur son prédécesseur.
 - **Process.xmi**
Modèle SimplePDL représentant un processus classique.
 - **Same_name.xmi**
Exemple de modèle contenant des doublons dans les noms.

- **SimplePDL.aird**
Fichier pour les vues graphiques associées à SimplePDL.
- **SimplePDL.ecore**
Définition du métamodèle EMF pour le langage SimplePDL.
- **SimplePDL.genmodel**
Modèle de génération de code EMF à partir de SimplePDL.ecore.
- **unvalidated.xmi**
Modèle contenant des erreurs intentionnelles pour tester la validation.
- **fr.n7.simplepdl2petri**
 - **simplepdltopetri.atl**
Transformation ATL du modèle SimplePDL vers un réseau de Petri.
- **fr.n7.simplePDL.edit**
- **fr.n7.simplePDL.editor**
- **fr.n7.simplepdl.exemples**
 - **created_simplePDL.xmi**
Modèle SimplePDL généré automatiquement à des fins de démonstration.
 - **ex1.pdl1**
Fichier source écrit dans la syntaxe PDL1, utilisé comme exemple.
 - **ex2.pdl2**
Fichier exemple exprimé avec le langage PDL2.
 - **ex3.pdl3**
Exemple de processus modélisé avec la syntaxe PDL3.
 - **My.simplepdl**
Modèle personnalisé au format SimplePDL utilisé pour des tests.
- **fr.n7.simplePDL.tests**
- **fr.n7.simplepdl.todot**
 - **src/fr.n7.simplepdl.todot.main/ToDot.java**
Classe Java principale pour la conversion du modèle SimplePDL au format DOT.
 - **src/fr.n7.simplepdl.todot.main/ToDot.mtl**
Fichier de métamodèle définissant la transformation SimplePDL vers DOT.
 - **model.dot**
Fichier de sortie DOT représentant le modèle SimplePDL.
 - **model.pdf**
Version PDF du modèle générée à partir du fichier DOT.
- **fr.n7.simplepdl.toHTML**
 - **src/fr.n7.simplepdl.toHTML.main/ToHTML.java**
Classe Java principale pour la transformation du modèle SimplePDL en page HTML.
 - **src/fr.n7.simplepdl.toHTML.main/ToHTML.mtl**
Fichier de métamodèle définissant la transformation SimplePDL vers HTML.
 - **model.html**
Fichier HTML généré représentant le modèle SimplePDL sous forme web.

- `fr.n7.taskmaster`
 - **TaskMaster.ecore**
Définition du métamodèle TaskMaster au format Ecore.
 - **TaskMaster.genmodel**
Fichier de génération permettant la création des classes Java à partir du métamodèle TaskMaster.
 - **test_process.xmi**
Modèle TaskMaster généré depuis un modèle PDF.
- `fr.n7.taskmasteratl`
 - **taskmaster.atl**
Transformation ATL du modèle SimplePDL vers un modèle TaskMaster.
- `fr.n7.taskmaster.edit`
- `fr.n7.taskmaster.editor`
- `fr.n7.taskmaster.tests`

9.2 Eclipse de déploiement

- `fr.n7.pdl1simplepdl.atl`
 - **pdl1tosimplepsl.atl**
Transformation ATL du modèle PDL1 vers un modèle SimplePDL.
- `fr.n7.simplepdl.design`
 - **simplepsl.odesign**
Définit la représentation graphique du langage SimplePDL dans Sirius, utilisée pour créer un éditeur visuel.
- `fr.n7.simplepdl.samples`
 - **developpement.simplepdl**
Modèle exemple conforme à SimplePDL, utilisé pour tester et illustrer l'éditeur graphique.
 - **representations.aird**
Contient les vues et diagrammes Sirius liés au modèle SimplePDL.